

Scaling Techniques Documentation

Numerical Variable:

I used MinMax scaling to normalize the numerical variables (tenure and MonthlyCharges) into a range between 0 and 1. The transformation was fitted on the training dataset and then applied to the test dataset to prevent data leakage. Only numerical features were scaled.

Code:

```
# MinMax Method
numeric_cols = ["tenure", "MonthlyCharges"]
scaler = MinMaxScaler()
# normalises all features to the [0,1] range

X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

# Only scaling numeric variable
X_train_scaled[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test_scaled[numeric_cols] = scaler.transform(X_test[numeric_cols])
```

Categorical variables:

Categorical variables were encoded to transform them into numerical formats suitable for machine learning models. Binary variables such as gender, Dependents, PhoneService, MultipleLines, InternetService, and Churn were converted using 0/1 mapping. Additionally, the Contract variable was encoded using ordinal encoding to reflect its inherent order, where longer contract durations correspond to higher values.

Code:

```
# Binary encoding
# Binary categorical variables were encoded using 0/1 mapping.
data["gender"] = data["gender"].map({"Female": 0, "Male": 1})
data["Dependents"] = data["Dependents"].map({"No": 0, "Yes": 1})
data["PhoneService"] = data["PhoneService"].map({"No": 0, "Yes": 1})
data["MultipleLines"] = data["MultipleLines"].map({"No": 0, "Yes": 1})
data["InternetService"] = data["InternetService"].map({
    "DSL": 0,
    "Fiber optic": 1
})
data["Churn"] = data["Churn"].map({"No": 0, "Yes": 1})

# Label encoding
# Label encoding was applied to ordinal categorical variables where a natural order exists.
contract_map = {
```

```
        "Month-to-month": 0,  
        "One year": 1,  
        "Two year": 2  
    }  
    data["Contract"] = data["Contract"].map(contract_map)  
  
    print("\nThe data after encoding:\n")  
    print(data)
```